Jarvis — a multimodal agent that operates the homelab

runs on the tailnet, never public
GPU node: desktop RTX 5080, on demand
acts only through Git — ArgoCD deploys

FIG. 01 CONTROL PLANE · FIG. 02 MODALITY OPTIONS · FIG. 03 DEPLOY FLOW · FIG. 04 FEATURE FLOW · FIG. 05 CORE & SERVING OPTIONS · FIG. 06 BUILD ORDER

01 Control plane

senses → intent → tools → git → cluster

The rule that keeps this safe and boring: Jarvis has no `kubectl apply` in its hands. It writes commits and opens PRs against `Home-Lab-Infra` and the app repos; ArgoCD stays the only thing that changes the cluster, so every action Jarvis takes is diffable and revertible.

A · SENSES / EDGE

Voice

Wake word on device, VAD, then a stream of audio frames. Mac mic, iPhone Shortcut, or a Pi/ESP32 satellite in another room.

Video

Webcam and screen capture. Frames are sampled, not streamed to the model — 1 fps on motion, plus a keyframe on request.

ns jarvis · k3s · UM790 (always on) + desktop agent (GPU, on demand)

deployed by ArgoCD like every other app

B · realtime gateway

one session per user · barge-in · <700 ms to first word

VAD + turn taking	STT (streaming)	TTS (streamed out)	frame & event ingest

Holds the media session and fuses the three modalities into one turn: transcript + latest keyframe + pending gesture event. Everything downstream sees plain JSON.

▼ ▼ ▼

E · WHAT IT ACTS ON

Home-Lab-Infra

PRs that bump image tags in `kustomization.yaml` or add a new `bootstrap/apps/*.yaml`.

App repos

tickup, trip-ai, and whatever is next. Feature branches and PRs only — CI still builds and tests.

Gesture

Hand landmarks classified on the edge into a small vocabulary. Emits discrete events (`confirm`, `cancel`, `next`), never raw video.

Text & hooks

Web console, Telegram bot, GitHub webhooks, cron. Same intent API as speech — voice is one client, not the architecture.

All clients are tailnet devices. Nothing listens on the router.

WEBRTC / WS

C · orchestrator deploy jarvis-core

Router
small talk → local model
code & planning → frontier model

Planner / graph
state machine, one node per step,
checkpointed so a run can resume

Policy gate
read = auto · write = PR
destructive = human confirm

Memory: recent turns in Redis, long-term facts and embeddings in Supabase Postgres + pgvector. Every run traced, so “why did it do that” is answerable.

D · tools (MCP) one server per capability

mcp-github
branch · commit · PR · read CI status

mcp-k8s (read-mostly)
pods · logs · events · rollout status

mcp-argocd
app health · sync · diff

mcp-scaffold yours
new app from the tickup template:
ns + ingress + kustomize + sealed secrets

mcp-coder
spawns a sandboxed k8s Job that runs a coding agent on a repo checkout

mcp-home optional
lights, WoL, notifications

GPU pool · node `desktop-5080` · **label** `gpu=true`
Woken by Wake-on-LAN when a GPU job is queued, drained and shut down when the queue empties. STT, TTS, the vision model and any local LLM are scheduled here with a toleration.

ArgoCD

Reconciles the merge. Jarvis then watches health and reports back in the same voice turn.

GHCR + Actions

Unchanged. The image Jarvis deploys is the image CI built.

Secrets never reach the model: the MCP servers hold tokens, the LLM only sees tool names and results.

02 Modality options

★ = what I'd start with

Each lane is swappable behind the gateway. Start on the cheap row, move to the GPU row when the latency annoys you.

Wake word + VAD

always listening, on device

★ **openWakeWord + Silero VAD**

Free, CPU-only, custom “Jarvis” model trainable in an afternoon. Runs on a Pi satellite.

Picovoice Porcupine

Best accuracy per CPU cycle, tiny SDKs everywhere. Free tier is per-user licensed.

Push-to-talk only

A hotkey or Shortcut. Zero false triggers, zero privacy questions — the honest starting point.

Speech → text

streaming, partials needed

★ **faster-whisper large-v3-turbo on the 5080**

Local, multilingual, ~2 GB VRAM at int8. Wrap it in a small FastAPI service with a streaming endpoint.

whisper.cpp on the UM790

No GPU wake needed for short commands; small/medium models only.

Deepgram / AssemblyAI streaming

200 ms partials with no infrastructure. Audio leaves the tailnet — fine for “deploy tickup”, not for everything.

Text → speech

first audio chunk fast

★ **Piper**

Real-time on CPU, dozens of voices incl. Italian. Robotic but instant, which suits a lab assistant.

Kokoro-82M or XTTS-v2

Much better prosody; XTTS clones a voice from 30 s of audio. GPU, so it pins the desktop awake.

ElevenLabs / OpenAI TTS

The “it actually sounds like Jarvis” option, metered per character.

Realtime pipeline

the thing that glues the turn together

★ **Pipecat**

Python, pluggable STT/LLM/TTS, interruption handling solved. Swap providers per env var.

LiveKit Agents

Self-hostable SFU + agent framework. Best if you want video and multiple rooms/devices properly.

Own WebSocket loop

~300 lines, no dependency risk, and you will re-implement barge-in yourself.

Vision	“look at this”, screen reading
★ Qwen2.5-VL 7B on the 5080	Strong on UI screenshots, charts and dashboards. Served over vLLM with an OpenAI-shaped API.
Moondream2 / MiniCPM-V	Tiny, always-resident. Good enough for presence, “is someone at the door”, gesture context.
Frontier vision API	For the hard one-offs: read this stack trace off my screen and open the PR.

Gesture	confirm / cancel / next / volume
★ MediaPipe Hands in the browser (WebGPU)	21 landmarks at 30 fps on the client; a 50-sample KNN over normalized landmarks covers six gestures. Only the event crosses the network.
MediaPipe Holistic in a GPU pod	Pose + hands + face for room-scale control. Needs the camera stream server-side.
Wearable instead of camera	Apple Watch double-tap / crown as the confirm channel. Works in the dark, no camera in the house.

03 Flow — “Jarvis, deploy TickUp”

spoken → merged → healthy, ~2 min

01	02	03	04 GATE	05	06	07
Heard	**Resolved**	**Spoken back**	**You confirm**	**PR**	**ArgoCD syncs**	**Reports / reverts**
Wake word, transcript, intent `deploy(app)`.	Reads GHCR for the newest green tag and the tag currently in Git.	“Moving tickup-backend from a3f1 to c92d, 4 commits. Confirm?”	Voice “yes”, a thumbs-up gesture, or the button in the notification.	One-line change to `kustomization.yaml`, auto-merged if checks pass.	Jarvis polls app health and the rollout, not the pod list.	Healthy → says so. Failing probes → offers to revert the commit.

04 Flow — “add CSV export to TickUp”

the beast mode one: Jarvis writes code in a sandbox

01	02	03	04	05 GATE	06
Spec turn	**Job spawned**	**Coding agent**	**PR + preview**	**You review**	**Ships**
Asks 2–3 clarifying questions out loud, writes a short spec into the issue.	k8s Job: repo checkout, no cluster creds, egress limited to GitHub + the model API.	Claude Code, Aider or OpenHands in the container. Runs the test suite itself.	Branch pushed, CI builds the image, an optional preview ns gets its own tailnet URL.	Jarvis narrates the diff. Merge is yours — this gate never gets automated.	Falls into flow 03 at step 05 — same deploy path as everything else.

A brand-new app (“Jarvis, spin up a new project called notes-ai”) is the same flow with `mcp-scaffold` in front: repo from template, then namespace, Ingress, kustomization and a sealed secret stub committed to `apps/notes-ai` plus an entry in `bootstrap/apps/`.

05 Core, serving and memory — pick one per row

all three rows are reversible later

agent core	★ FastAPI + LangGraph Explicit graph, Postgres checkpointer, human-in-the-loop interrupts built in. Matches the FastAPI stack you already deploy.	Pydantic AI Lighter and typed end to end; less machinery for long-running resumable runs.	Plain tool-use loop One while-loop over the messages API. Fewest concepts, and you own retries, state and tracing.
brain / serving	★ hybrid via LiteLLM One OpenAI-shaped endpoint; frontier model for planning and code, local Qwen/Llama for routing, summaries and offline fallback.	Local only — vLLM on the 5080 16 GB VRAM fits a 14B at 4-bit comfortably. Private and free per token; weaker at multi-file code edits.	Ollama Fastest to stand up, model pull as a one-liner. Lower throughput than vLLM under concurrent modalities.
memory & traces	★ Supabase + pgvector, Redis for the session Reuses the database you already run outside the cluster. No new stateful workload on the node.	Qdrant in-cluster Better recall tooling and filtering; adds a PVC and a backup story to a single-node cluster.	Langfuse for traces Self-hosted run/cost/latency history — pair with either memory choice, and with the Grafana stack on the roadmap.

06 Build order

each phase is useful on its own

P0 · START HERE

Hands, no face

jarvis-core + mcp-github + mcp-argocd, text only, driven from a web console. Prove the deploy flow end to end before any audio exists.

P1

Voice

Push-to-talk → Pipecat + whisper + Piper. Wake word once you trust the confirm gate.

P2

GPU on demand

Desktop joins as a k3s agent, nvidia device plugin, WoL wake and queue-driven scale to zero. Local STT/TTS/VLM move here.

P3

Vision + gesture

Screen and camera into the turn; MediaPipe events as the confirm/cancel channel. Six gestures, not thirty.

P4

Autonomy

mcp-coder writing features, preview namespaces per PR, and Jarvis watching Grafana/Loki to open its own issues.

OPEN QUESTIONS

Which gate is truly manual — merges only, or every write? Whether audio may leave the tailnet at all (that single answer decides half of fig. 02). And whether the desktop is allowed to boot itself at 3 a.m., or Jarvis is simply text-only while the GPU sleeps.